ADAPTIVE GAMER COACHING SYSTEM WITH RAGE-QUIT AND ADDICTION PREDICTION

A project report submitted in partial fulfillment of the requirements for the completion of application development in

Bachelor of Technology

In

Computer Science and Engineering

By

(2411CS010003) Mohammed Abdullah

(2411CS010005) Adi Thanusri

(2411CS010006) Aditi Singh

(2411CS010010) Akkati Keerthi Priya

Under the guidance of

Mrs. S. Sowmya



Department of Computer Science & Engineering

MALLA REDDY UNIVERSITY, HYDERABAD

2025-2026

MALLA REDDY UNIVERSITY
(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

CERTIFICATE

This is to certify that this is the Application development lab record entitled

ADAPTIVE GAMER COACHING SYSTEM WITH RAGE-QUIT AND ADDICTION PREDICTION

This is to certify that this is the Application development lab record entitled "**Adaptive Gamer Coaching System with Rage-Quit and Addiction Prediction**", submitted by **Mohammed Abdullah (2411CS010003)**, **Adi Thanusri (2411CS010005)**, **Aditi Singh (2411CS010006)**, **Akkati Keerthi Priya (2411CS010010)**, B.Tech II year II semester, Department of CSE during the year 2025-2026. The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma.

Internal Guide
Mrs. S. Sowmya

DEAN CSE
Dr. SHAIK MEERAVALI

External Examiner

DECLARATION

I declare that this project report titled “Adaptive Gamer Coaching System with Rage-Quit and Addiction Prediction” submitted in partial fulfillment of the degree of B. Tech in CSE is a record of original work carried out by us under the supervision of Mrs. S. Sowmya, and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Mohammed Abdullah	**[2411CS010003]**
Adi Thanusri	**[2411CS010005]**
Aditi Singh	**[2411CS010006]**
Akkati Keerthi Priya	**[2411CS010010]**

ACKNOWLEDGEMENT

It gives us immense pleasure to express our profound gratitude to all those who extended their valuable support, encouragement and guidance in the successful completion of this project.

First and foremost, we would like to express our heartfelt thanks to Dr. V. S. K. Reddy, Hon'ble Vice-Chancellor of our esteemed institution, for providing an excellent academic environment, the necessary facilities and constant encouragement that motivated us to pursue this project with enthusiasm and dedication.

We are deeply indebted to Dr. SHAIK MEERAVALI, Dean, Department of Computer Science and Engineering, for his valuable support, expert advice and continuous encouragement throughout the course of this project.

We wish to express our sincere appreciation and heartfelt gratitude to our project guide Mrs. S. Sowmya, for her invaluable guidance, constructive suggestions and constant supervision during every phase of this project. Her technical expertise and insightful feedback were instrumental in shaping this work and bringing it to completion.

We also extend our thanks to all the faculty members of the Department of Computer Science and Engineering for their cooperation and assistance whenever required.

Finally, we express our deep gratitude to our parents, friends and well-wishers for their unwavering support, patience and motivation throughout this journey.

Mohammed Abdullah **[2411CS010003]**

Adi Thanusri **[2411CS010005]**

Aditi Singh **[2411CS010006]**

Akkati Keerthi Priya **[2411CS010010]**

ABSTRACT

Competitive multiplayer games and esports have changed how people play. Longer sessions and more intense, emotional matches bring benefits such as enjoyment and skill development, but they can also bring stress, anger after losses, rage-quitting, disturbed sleep and gaming addiction. Current gaming analytics focus on performance numbers such as kills and win rate, and say little about the emotional and behavioural effects of play on the player.

The Adaptive Gamer Coaching System is a web application that predicts a player's **tilt (rage-quit) probability** and **gaming-addiction level** from thirteen behavioural and wellbeing factors: play time, sessions, late-night play, sleep, toxic lobbies, spending, stress, anxiety, anger after losses, low mood, happiness, loneliness and offline social life. Two deep neural networks (Multi-Layer Perceptrons, Dense-128 → Dense-64) are trained on 22,000 synthetic, population-shaped profiles; the tilt model reaches 79.7% accuracy with an ROC-AUC of 0.86 (close to the 79.3% noise ceiling of the data) and the addiction model reaches 92.6% accuracy with a macro F1-score of 0.91.

What sets the system apart is that it explains and coaches. Sampled Shapley values show how many points each habit adds to tilt and to addiction, and a coaching engine turns the result into a plain-language verdict, a phased recovery plan with a week-by-week step-down schedule, a projected outcome that re-runs the same models on the plan's targets, WHO-based warning signs and free support resources. A searchable database of 144 synthetic Valorant players makes the system easy to explore.

The back end is built with FastAPI and scikit-learn and the front end with React and Vite. The framework joins artificial intelligence, behavioural science and gaming analytics to support casual gamers and future esports professionals in keeping their play healthy.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
1	**Introduction**	**1**
	1.1 Summary of Project	1
	1.2 Problem Statement	1
	1.3 Objective of the Project	2
	1.4 Goal of Project	2
2	**Problem Identification**	**4**
	2.1 Existing System	4
	2.2 Proposed System	5
3	**Software and Hardware Requirements**	**7**
	3.1 Software Requirements	7
	3.2 Hardware Requirements	7
4	**Design and Implementation**	**8**
	4.1 System Architecture	8
	4.2 Implementation	10
5	**Code**	**14**
	5.1 Coding	14
	5.2 Screenshots	24
6	**Results & Conclusion**	**30**
	6.1 Results	30
	6.2 Conclusion	31
7	**References**	**33**

LIST OF FIGURES

FIGURE NO.	NAME OF THE FIGURE	PAGE NO.
Fig 1	System Architecture	8
Fig 2	Data Flow Diagram	9
Fig 3	Modular Interconnectivity Framework	9
Fig 4	Home page with player search	24
Fig 5	Player profile and report overview	24
Fig 6	Verdict lower-third	25
Fig 7	Tilt / rage-quit score	25
Fig 8	What is driving the score (factor board)	27
Fig 9	Now versus after the plan	27
Fig 10	Recovery plan and step-down schedule	27
Fig 11	Warning signs and support	28
Fig 12	Behaviour and check-in sliders	29
Fig 13	Training loss curves for the two networks	31

LIST OF TABLES

Table No.	Table Name	Page No.
1	Comparative Analysis of Existing Research	4
2	Software Requirements	7
3	Hardware Requirements	7
4	Gaming Behaviour and Mental Health Feature Set	10
5	Training Configuration	11
6	API Endpoints	13
7	Model Performance Evaluation	30

CHAPTER-1
INTRODUCTION

1.1 Summary of Project

Competitive multiplayer games and esports have changed how millions of people spend their time. Players now spend long hours in emotionally intense matches, and for a growing number of them that intensity turns into stress, anger after losses, broken sleep and, in some cases, a gaming habit that starts to crowd out the rest of life. Existing gaming analytics tools measure what happens inside the match, such as kills, deaths and win rate, but say very little about what is happening to the person playing.

The Adaptive Gamer Coaching System is a web application that closes that gap. A user searches for a Valorant player by Riot ID, or types in their own numbers, and the system reads thirteen behavioural and wellbeing factors: how long and how late they play, how much they sleep, how stressed, anxious and angry they feel, how isolated they are and how much they spend. Two deep neural networks (Multi-Layer Perceptrons) then predict the player's **tilt (rage-quit) probability** and their **gaming-addiction level** (Low, Medium or High).

The system does not stop at a score. For every prediction it uses Shapley-value attribution to show which habits are pushing the risk up and by how many points, it writes a plain-language verdict, builds a phased recovery plan (Today, Weeks 1–2, Weeks 3–6) with a week-by-week step-down schedule for daily play time, and re-runs the same models on the plan's targets to show the predicted improvement. It also lists warning signs drawn from the WHO description of gaming disorder and free support helplines.

The back end is built with FastAPI and scikit-learn; the front end is a React 19 and Vite application. The models are trained on 22,000 synthetic, population-shaped player profiles, and the player database that powers the search contains 144 synthetic Valorant players across six regions, so the whole system can be explored without any personal data.

1.2 Problem Statement

Online games generate enormous amounts of telemetry, yet analysts and tools concentrate on results: wins versus losses, kills versus deaths, rank movement. They pay little attention to the psychological state of the person behind the account. A player who is “tilted” after a losing streak, or who is quietly sacrificing sleep, study and relationships for one more match, gets no feedback at all until the damage is already done. Tilt harms the player's own results and the experience of everyone in the lobby, and an unnoticed gaming habit can harm personal wellbeing, academic or professional performance and social life.

Research on gaming behaviour has produced useful findings, but three gaps keep that research from helping an ordinary player. First, most predictive models are black boxes: they output a risk level without saying which habits caused it, so the player does not know what to change. Second, tools that do give advice give the same generic advice to everyone, such as “take a break”, rather than numbers that match the individual. Third, existing work treats addiction, tilt and performance separately instead of looking at the whole behavioural picture.

There is therefore a need for an integrated, explainable system that predicts tilt and addiction risk from everyday behaviour, explains the prediction in terms the player can act on, and turns it into a concrete, personalised plan the player can follow.

1.3 Objective of the Project

The primary objective of this project is to design and build a web-based coaching system that predicts the emotional and behavioural risks of competitive gaming and helps the player act on them. The specific objectives are:

- **Predict tilt (rage-quit) risk.** Train a deep neural network on nine inputs (stress, anxiety, anger after losses, daily hours, toxic lobbies, late-night play, weekly sessions, sleep and loneliness) to output the probability that the player is prone to rage-quitting.
- **Classify gaming addiction.** Train a second network on ten inputs (daily hours, sessions, late-night play, sleep, loneliness, offline social life, spending, years of play, happiness and low mood) to classify the player as Low, Medium or High addiction.
- **Explain every prediction.** Use Shapley-value attribution against a healthy baseline so that each habit shows how many points it adds to tilt and to addiction.
- **Coach with the player's own numbers.** Generate a verdict, per-factor advice, a phased recovery plan, a step-down schedule and a projected outcome that re-runs the same models on the plan's targets.
- **Make the data feel real.** Provide a searchable Valorant player database with rank, stats, agents, recent matches and play-time pattern, so a Riot ID produces a complete profile and report.
- **Present it well.** Build a fast, readable, responsive interface that explains the result without overwhelming the reader.

1.4 Goal of Project

The goal of the project is to help gamers stay in control of the game rather than the other way round. The system aims to act as a coach that sits beside a player, reads their habits honestly, and says in plain language what is going well, what is putting them at risk and what to do about it, starting today.

A key goal is explainability: every number on screen can be traced to the habits behind it, and every piece of advice uses the player's own values. A second goal is actionability: the output is a plan with targets, a taper schedule and a projected result, not just a label. A third goal is safety and honesty: the system is a screening aid, never a diagnosis, and it points to professional help and free helplines whenever the warning signs call for it.

Ultimately, the goal is to demonstrate that machine learning, behavioural science and game analytics can be combined into a practical tool for casual players and for the next generation of esports professionals, one that is ready to be connected to live game data such as the Riot Games API in future work.

CHAPTER-2
PROBLEM IDENTIFICATION

2.1 Existing System

Today's gaming platforms and third-party trackers provide match history, rank progression, agent statistics and heat-maps. These products are very good at describing performance. They do not describe wellbeing. A separate body of academic research studies tilt, emotion regulation, gaming disorder and their links to sleep, mood and loneliness, using psychometric scales, clinical surveys, behavioural logs, text mining and even neuroimaging. Table 1 summarises the main approaches reviewed for this project.

Table 1: Comparative Analysis of Existing Research in Gaming Behaviour and Addiction Detection

Reference	Methodology	Accuracy	Advantages	Disadvantages
Cregan et al. (2024)	Online study (N=1,007) using SEQ and BERQ psychometric scales	33.30% of tilt variance explained	Globally validated instruments with a large sample	Relies on self-reporting and recall bias
Alan et al. (2025)	ML framework analysing user comments and text-based patterns	92.49% (XGBoost)	Non-intrusive data collection; easily scalable	Excludes visual cues; limited to users who write text
Knol et al. (2021)	Behavioural logging in custom games plus real-time surveys	60.10%	High model explainability for designers	Lower accuracy than clinical models; limited genres
Ahmed et al. (2024)	Clinical survey using the Lemmens Gaming Addiction Scale	93.00% (AdaBoost)	High predictive accuracy for demographic risk	Limited geographical generalisability; small dataset
Weinstein (2010)	PET imaging of dopamine D2 receptor binding	99.90% confidence level	Objective, biological evidence	Prohibitive equipment cost; small sample
Beck & Gong (2013)	Analysis of 390,148 events across 34 game levels	78.00%	Allows proactive, early intervention	Needs feature engineering per level
Rismayanti (2024)	Kaggle dataset evaluated with 5-fold cross-validation	84.27%	Handles categorical and numerical data well	Limited by the static nature of historical datasets

As Table 1 shows, accuracies range from about 60% for explainable behavioural models to 93% for clinical-survey models, but each approach carries a cost: self-report bias, equipment cost, small samples or dependence on one game level. The following drawbacks of the existing approaches motivate this project:

- **The black box of prediction.** Models predict well but explain little. Few works say which gameplay and lifestyle factors cause the predicted addiction or frustration.
- **Generic intervention design.** Interventions are rarely personalised, and there is little evidence on which design works best for which player.
- **No integrated behavioural framework.** Addiction, tilt and performance are studied separately, not as one connected picture.
- **Limited longitudinal validity.** Most studies are cross-sectional snapshots rather than long-term observation of behaviour in the real world.
- **Ethics and privacy.** Monitoring behaviour raises privacy concerns that tools must address by design.
- **Fragmented tracking.** Players have no single place that combines their play data with sleep, mood and social context.

2.2 Proposed System

The proposed system is an explainable coaching platform built around two deep neural networks. Instead of reporting only a probability, it explains the probability, converts the explanation into a plan and shows the plan's likely effect. The design rests on five ideas.

1. One profile, thirteen factors. A player is described by thirteen behavioural and wellbeing factors grouped as habits (play time, sessions, late-night play, sleep, toxic lobbies, spending), mind (stress, anxiety, anger after losses, low mood, happiness) and social life (loneliness, offline social interaction). The same values drive both models.

2. Two Multi-Layer Perceptrons. A tilt network (9 inputs, Dense-128, Dense-64, probability output) and an addiction network (10 inputs, Dense-128, Dense-64, three-class output) are trained with the Adam optimiser and early stopping. The addiction output is also summarised as a 0–100 addiction index, the expected severity across Low, Medium and High.

3. Shapley-value explanations. For each prediction the coaching engine moves the profile from a healthy baseline to the player's values one factor at a time, in 160 random orderings, and records how much each step changes each model's output. The averaged contributions sum exactly to the gap between the player and the healthy baseline, so every factor gets an honest “points added” figure for tilt and for addiction.

4. A plan, not just a score. The engine writes a verdict (headline, one-line explanation, next action), per-factor advice using the player's numbers, a Today / Weeks 1–2 / Weeks 3–6 recovery plan, a week-by-week step-down for daily play time and a projected outcome that re-runs the same models on the plan's targets for the habits the player controls.

5. A searchable player database. A built-in database of 144 Valorant players generated from eight behavioural archetypes (balanced, weekend, grinder, tilted, night owl, escapist, collector, recovering) keeps rank, K/D, headshot rate, ACS, agents, the last ten matches and the hour-by-hour play pattern consistent with the wellbeing answers. Any other Riot ID gets a stable generated profile.

The system runs on standard hardware, answers a request in a fraction of a second, and is split cleanly into a React front end and a FastAPI back end so that a live game API can replace the player database without touching the models.

CHAPTER-3
SOFTWARE AND HARDWARE REQUIREMENTS

3.1 Software Requirements

Table 2: Software Requirements

Tool / Technology	Purpose
Python 3.10 – 3.13	Programming language for the machine learning pipeline and the back end
FastAPI + Uvicorn	REST API framework and ASGI server (port 8000)
scikit-learn 1.5.2	MLPClassifier neural networks, StandardScaler, evaluation metrics (version pinned for the saved models)
NumPy, joblib, Pydantic	Numerical computing, model serialisation, request validation
Node.js 20+	Runtime for the front-end toolchain
React 19 + Vite	Single-page front-end application and dev server (port 5173)
lucide-react	Icon set used across the interface
CSS (custom design system)	Broadcast-style visual design, animations, responsive layout
Git / GitHub	Version control and source hosting

3.2 Hardware Requirements

Table 3: Hardware Requirements

Component	Specification
Processor	Intel Core i5 (8th generation or newer) or AMD Ryzen 5
Memory (RAM)	8 GB
Storage	256 GB SSD (the project itself needs well under 1 GB)
System type	64-bit operating system (Windows 10/11, macOS or Linux)
Display	Any modern browser; the interface adapts from phone to desktop widths
Network	Not required to run locally; an internet connection is needed only to install dependencies

CHAPTER-4
DESIGN AND IMPLEMENTATION

4.1 System Architecture

The system follows a layered client–server design. The user works in a React single-page application. Every search or slider change is sent as JSON to a FastAPI service, which validates the input, scales it, runs the two trained neural networks and hands the results to the coaching engine. The engine returns one report object containing the scores, the Shapley drivers, the plan and the projected outcome. The player database and the trained models are plain files loaded once at start-up, so the service is stateless and fast.

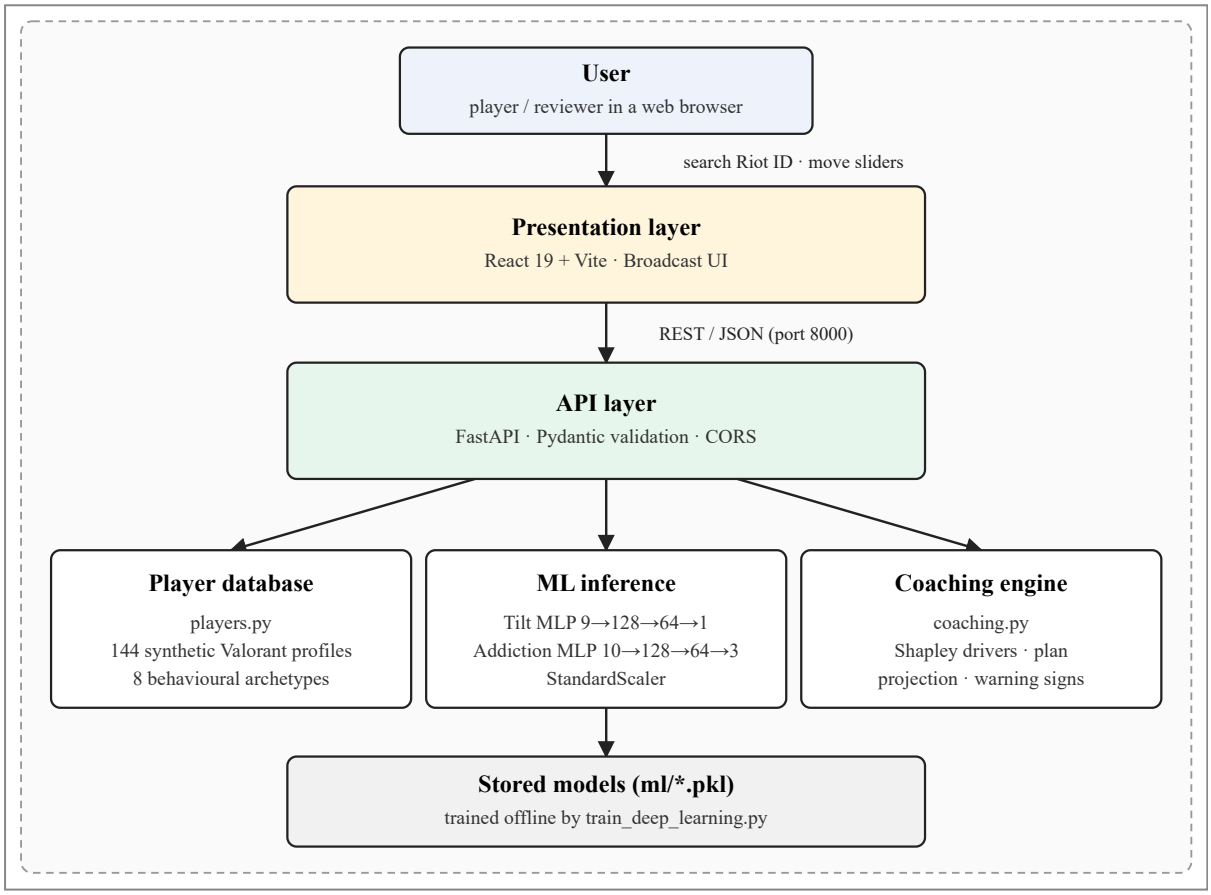


Fig 1: System Architecture

Figure 2 shows how data moves through the system. A Riot ID is resolved against the player database (or the user adjusts the sliders by hand), the thirteen inputs are validated and scaled, the stored models produce the predictions, and the coaching engine turns them into the report that the front end renders.

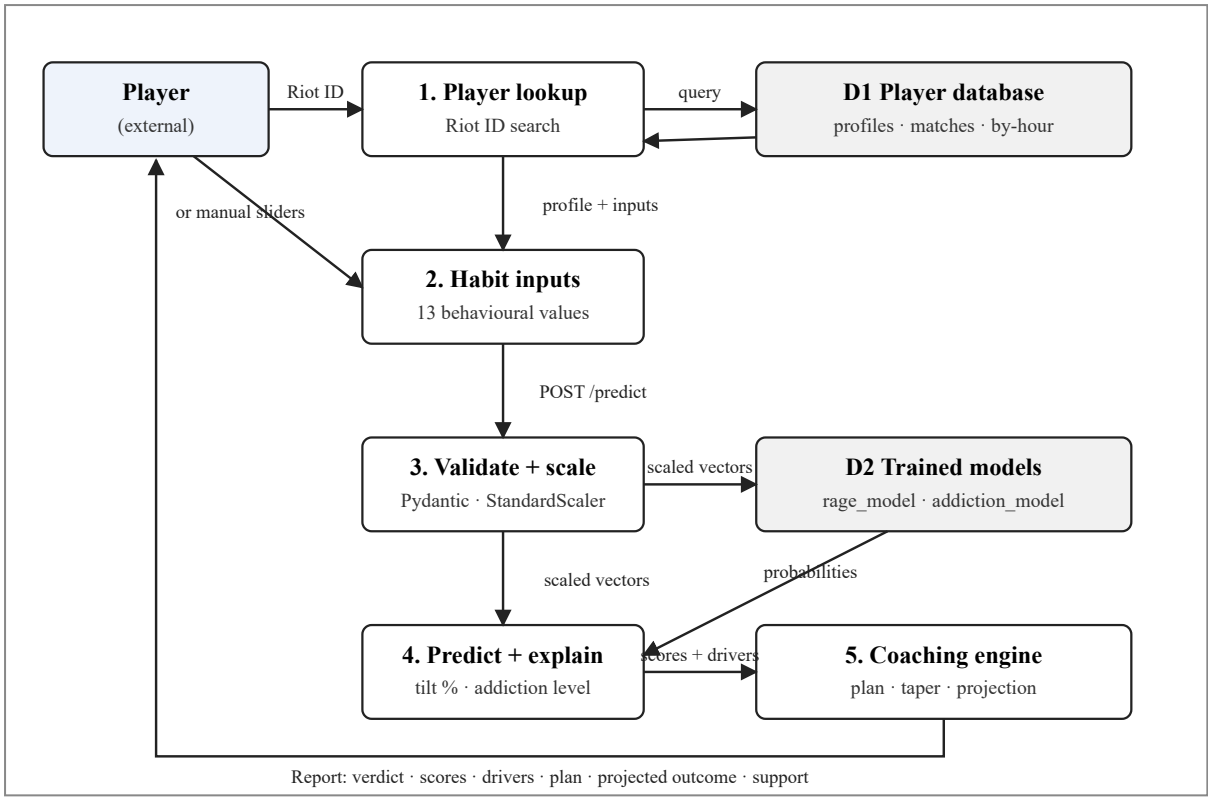


Fig 2: Data Flow Diagram

Module connectivity

The operational flow is divided into specialised modules, shown in Figure 3: the user-interface module, the data-collection module, the data-processing module, the feature module, the machine-learning prediction module, the risk-evaluation module and the adaptive-coaching module.

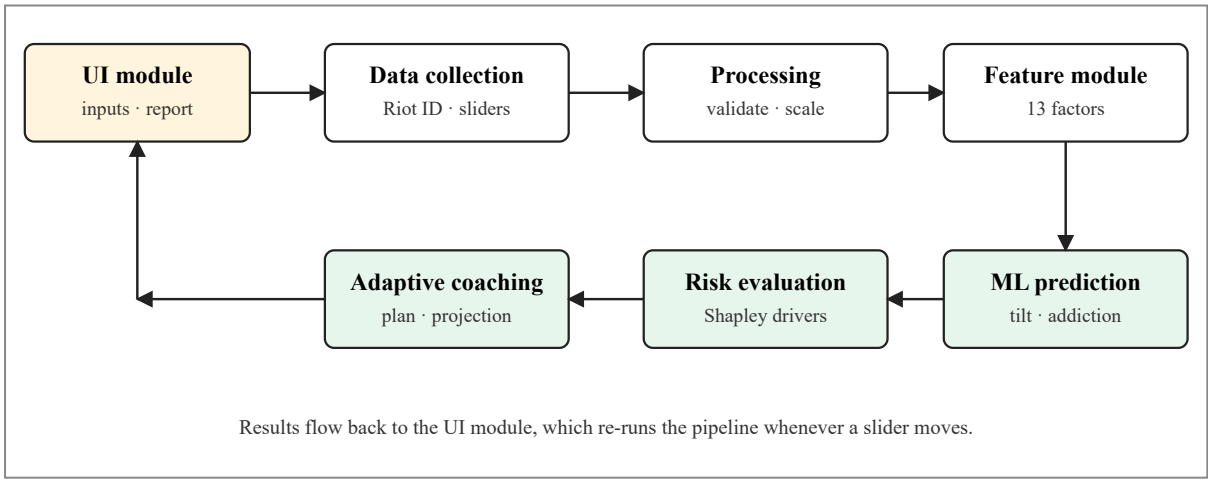


Fig 3: Modular Interconnectivity Framework

- **User-interface module:** player search, the “enter your own numbers” panel, the player card and the report sections.
- **Data-collection module:** reads a profile from the player database or collects the thirteen slider values.
- **Data-processing module:** validates ranges with Pydantic and standardises the values with the saved scalers.
- **Feature module:** maps the thirteen factors to the tilt model (nine inputs) and the addiction model (ten inputs).
- **ML prediction module:** runs the two MLPs and returns the tilt probability and the Low / Medium / High addiction class with its probabilities.
- **Risk-evaluation module:** computes Shapley contributions for every factor and compares each value with its healthy range.
- **Adaptive-coaching module:** writes the verdict, advice, plan, taper schedule, warning signs, support resources and projected outcome.

4.2 Implementation

4.2.1 Dataset

The models are trained on a synthetic dataset of 22,000 behavioural profiles. Sixteen thousand are drawn from population-shaped distributions (for example daily play from a gamma distribution, sleep from a normal distribution centred on seven hours, toxic exposure from a beta distribution) and six thousand are drawn uniformly so that extreme players are also covered. Labels are generated from documented risk functions: the tilt label is sampled from a logistic function of emotional load, provocation and fatigue, and the addiction label comes from a weighted score loosely following the warning signs of gaming disorder (time and frequency, late-night play, sleep loss, social withdrawal, spending and low mood), cut into Low, Medium and High bands. Because every input the user can change has a defined effect in the risk functions, the dashboard behaves sensibly when a slider is moved. The data is split 80% for training (17,600 rows) and 20% for testing (4,400 rows).

Table 4: Gaming Behaviour and Mental Health Feature Set

Feature	Description	Used by
daily_gaming_hours	Average hours played per day	Tilt, Addiction
weekly_sessions	Gaming sessions per week	Tilt, Addiction
night_gaming_ratio	Share of play that starts after 10 PM	Tilt, Addiction
sleep_hours	Average sleep per night	Tilt, Addiction

stress_level	Self-reported stress (1–10)	Tilt
anxiety_score	Self-reported anxiety (0–10)	Tilt
aggression_score	Anger after losses (0–10)	Tilt
toxic_exposure	Share of lobbies with abusive players	Tilt
loneliness_score	Loneliness (0–10)	Tilt, Addiction
social_interaction_score	Offline social life (0–10)	Addiction
microtransactions_spending	Monthly in-game spending (USD)	Addiction
years_gaming	Years the player has gamed	Addiction
happiness_score	Happiness (0–10)	Addiction
depression_score	Low mood (0–10)	Addiction

4.2.2 Neural network models

Both models are scikit-learn *MLPClassifier* networks with the same hidden structure, Dense-128 followed by Dense-64 with ReLU activation, L2 regularisation (alpha = 0.001), the Adam optimiser and early stopping. Inputs are standardised with a *StandardScaler* fitted on the training split only. The tilt network has a single sigmoid output that gives the rage-quit probability, which is shown as Low (below 40%), Medium (40–70%) or High (70% and above). The addiction network has a three-way softmax output; its class probabilities are also combined into a 0–100 **addiction index** using the expected severity (Low = 0, Medium = 0.5, High = 1). Training stopped after 23 epochs for the tilt model and 19 epochs for the addiction model.

Table 5: Training Configuration

Parameter	Tilt (rage-quit) MLP	Addiction MLP
Input features	9	10
Hidden layers	Dense-128 → Dense-64 (ReLU)	Dense-128 → Dense-64 (ReLU)
Output	Probability (sigmoid)	Low / Medium / High (softmax)
Optimiser	Adam, learning rate 0.001	Adam, learning rate 0.001
Regularisation (L2 alpha)	0.001	0.001
Early stopping	Yes (10 epochs without improvement)	Yes (10 epochs without improvement)
Train / test split	80% / 20% (stratified)	80% / 20% (stratified)

Random state	42	42

4.2.3 Explaining the prediction with Shapley values

A prediction is only useful if the player can act on it, so the coaching engine explains every score. It uses a sampled Shapley-value method. A “healthy baseline” profile is defined (for example 2 hours of play a day, 8 hours of sleep, low stress). For 160 random orderings of the thirteen factors, the profile is moved from the baseline to the player's values one factor at a time, and the change in each model's output at each step is credited to that factor. Averaged over the orderings, the credits sum exactly to the difference between the player's score and the baseline score:

$$\Sigma_i \varphi_i = f(\text{player}) - f(\text{healthy baseline})$$

This property keeps the explanation meaningful even when a model saturates near 0% or 100%. The result is shown as “points added to tilt” and “points added to addiction” for each factor, together with the factor's value, its healthy range and its status (healthy, watch or at risk).

4.2.4 Coaching engine

The coaching engine (*backend/coaching.py*) converts the numbers into guidance. It chooses a headline and a one-line explanation from five cases (both alarms on, gaming taking over, tilt running the show, warning zone, healthy balance); picks the top two drivers of risk; writes advice for each factor that quotes the player's own value; builds a three-phase plan (Today, Weeks 1–2, Weeks 3–6); creates a taper schedule that steps daily play down towards three hours; re-runs the models on the plan's targets for the habits the player directly controls (play time, sessions, late-night play, sleep, toxic lobbies and spending) to project the improvement; checks five warning signs adapted from the WHO description of gaming disorder; and adds free support resources, shown prominently when the signs are strong. The verdict panel displays this as a headline, one deck line and three ruled lines (Now, Driving it, With the plan) so that the main message can be read in seconds.

4.2.5 Player database

The module *backend/players.py* generates a database of 144 Valorant players across North America, Europe, Asia-Pacific, Korea, Brazil and Latin America. Each player is created from one of eight behavioural archetypes, which fixes plausible ranges for the thirteen wellbeing factors, and then rank, account level, K/D, headshot rate, ACS, win rate, agent pool, the last ten matches (including surrenders) and the hour-by-hour play pattern are derived from the same archetype, so the numbers tell one consistent story. Profiles are seeded from the Riot

ID, so a search always returns the same player, and an unknown Riot ID receives a stable generated profile.

4.2.6 REST API

Table 6: API Endpoints

Endpoint	Purpose	Main output
GET /health	Model status check	status, models_loaded, architecture
GET /players/search?q=	Search players by Riot ID (name or name#tag)	Up to 8 matching players with rank and region
GET /players/lookup?riot_id=	Full profile for a Riot ID	Rank, stats, agents, matches, play-time pattern, 13 inputs
POST /predict	Run both models and the coaching engine	rage_probability, addiction_category, addiction_index, report (summary, drivers, plan, taper, projected, warning_signs, support)

4.2.7 Front end

The interface is a React 19 single-page application styled with a custom “broadcast package” design system: a dark stage, a light lower-third panel for the verdict, signal red, field green and amber for risk levels, and condensed display type. The start screen offers player search and a manual-entry option. After a player is chosen, the page shows the player card, the verdict, the two scores, the factor board, a head-to-head comparison of “now” versus “after the plan”, the recovery plan and the support section. The report re-runs automatically while sliders are dragged. A short load-in animation, staggered section entrances, number count-ups and scroll reveals add polish, and all motion is switched off for users who prefer reduced motion. The layout stacks cleanly on phone widths.

CHAPTER-5
CODE

5.1 Coding

train_deep_learning.py

```
"""
Adaptive Gamer Coaching System — Deep Neural Network Training Pipeline
Trains Scikit-Learn Multi-Layer Perceptron (MLP) models for Rage-Quit (tilt) and
Addiction prediction, with loss-curve tracking and held-out evaluation metrics.
Pure NumPy & Scikit-Learn implementation (no pandas needed).

Run from the ml/ folder:  python train_deep_learning.py

Training data is SYNTHETIC. Labels come from documented risk functions (below) built
from the behavioural factors the dashboard asks about, plus random noise, so that every
input the user can change has a real, explainable effect on the prediction.
"""

import numpy as np
import joblib
import json
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score, f1_score

print("=" * 70)
print("ADAPTIVE GAMER COACH — DEEP NEURAL NETWORK (MLP) PIPELINE")
print("=" * 70)

RAGE_FEATURES = [
    'stress_level', 'anxiety_score', 'aggression_score', 'daily_gaming_hours', 'toxic_exposure',
    'night_gaming_ratio', 'weekly_sessions', 'sleep_hours', 'loneliness_score'
]

ADDICTION_FEATURES = [
    'daily_gaming_hours', 'weekly_sessions', 'night_gaming_ratio', 'sleep_hours',
    'loneliness_score', 'social_interaction_score', 'microtransactions_spending',
    'years_gaming', 'happiness_score', 'depression_score'
]

rng = np.random.default_rng(42)
N_REALISTIC = 16000   # population-shaped samples
N_UNIFORM = 6000      # uniform samples so extremes are covered too
N = N_REALISTIC + N_UNIFORM

print(f"\n[1/4] Generating training data ({N} samples)...")


def mix(realistic, lo, hi, integer=False):
    uniform = rng.uniform(lo, hi, N_UNIFORM)
    out = np.clip(np.concatenate([realistic, uniform]), lo, hi)
    return np.round(out) if integer else out


X = {
    'stress_level':              mix(rng.normal(5, 2, N_REALISTIC), 1, 10),
    'anxiety_score':             mix(rng.normal(4.5, 2.2, N_REALISTIC), 0, 10),
    'aggression_score':          mix(rng.normal(4.5, 2.2, N_REALISTIC), 0, 10),
    'daily_gaming_hours':        mix(rng.gamma(2.0, 1.8, N_REALISTIC), 0, 16),
    'toxic_exposure':            mix(rng.beta(2, 4, N_REALISTIC), 0, 1),
    'night_gaming_ratio':        mix(rng.beta(2, 3.5, N_REALISTIC), 0, 1),
    'weekly_sessions':           mix(rng.normal(15, 8, N_REALISTIC), 1, 50, integer=True),
    'sleep_hours':               mix(rng.normal(7, 1.3, N_REALISTIC), 3, 12),
```

```
    'loneliness_score':           mix(rng.normal(4.5, 2.3, N_REALISTIC), 0, 10),
    'social_interaction_score':   mix(rng.normal(5.5, 2.2, N_REALISTIC), 0, 10),
    'microtransactions_spending': mix(rng.gamma(1.2, 30, N_REALISTIC), 0, 500),
    'years_gaming':               mix(rng.normal(7, 4, N_REALISTIC), 0, 30, integer=True),
    'happiness_score':            mix(rng.normal(6, 2, N_REALISTIC), 0, 10),
    'depression_score':           mix(rng.normal(3.5, 2.2, N_REALISTIC), 0, 10),
}


def sigmoid(z):
    return 1 / (1 + np.exp(-z))


# ── Rage-quit (tilt) label ──
# Emotional load + provocation + fatigue. Sleep debt and toxic lobbies matter as much
# as stress itself; loneliness and long sessions add smaller effects.
rage_logit = (
    0.45 * (X['stress_level'] - 5)
    + 0.25 * (X['anxiety_score'] - 5)
    + 0.35 * (X['aggression_score'] - 5)
    + 2.2 * (X['toxic_exposure'] - 0.3)
    + 1.2 * (X['night_gaming_ratio'] - 0.3)
    + 0.12 * (X['daily_gaming_hours'] - 4)
    + 0.45 * (7 - X['sleep_hours'])
    + 0.15 * (X['loneliness_score'] - 5)
    + 0.02 * (X['weekly_sessions'] - 15)
    - 1.0
)
y_rage = (rng.uniform(0, 1, N) < sigmoid(rage_logit)).astype(int)

# ── Addiction label (3 classes) ──
# Loosely follows gaming-disorder warning signs: time & frequency, late-night play,
# sleep loss, social withdrawal, spending, and gaming used to escape low mood.
addiction_score = (
    0.45 * X['daily_gaming_hours']
    + 0.06 * X['weekly_sessions']
    + 2.0 * X['night_gaming_ratio']
    + 0.35 * np.maximum(0, 7 - X['sleep_hours'])
    + 0.15 * X['loneliness_score']
    - 0.15 * X['social_interaction_score']
    + 0.006 * X['microtransactions_spending']
    + 0.03 * X['years_gaming']
    - 0.15 * X['happiness_score']
    + 0.15 * X['depression_score']
    + rng.normal(0, 0.5, N)
)
y_addiction_str = np.where(addiction_score < 4.5, 'Low', np.where(addiction_score < 7.5, 'Medium', 'High'))

print(f"  Rage-quit positive rate: {y_rage.mean():.1%}")
for cls in ['Low', 'Medium', 'High']:
    print(f"  Addiction {cls:<6}: {(y_addiction_str == cls).mean():.1%}")

X_rage = np.column_stack([X[f] for f in RAGE_FEATURES])
X_addiction = np.column_stack([X[f] for f in ADDICTION_FEATURES])

le = LabelEncoder()
y_addiction = le.fit_transform(y_addiction_str)

Xr_train, Xr_test, yr_train, yr_test = train_test_split(X_rage, y_rage, test_size=0.2, random_state=42,
stratify=y_rage)
Xa_train, Xa_test, ya_train, ya_test = train_test_split(X_addiction, y_addiction, test_size=0.2,
random_state=42, stratify=y_addiction)

scaler_rage = StandardScaler().fit(Xr_train)
scaler_addiction = StandardScaler().fit(Xa_train)


def make_mlp():
    return MLPClassifier(
        hidden_layer_sizes=(128, 64),
```

```
        activation='relu',
        solver='adam',
        alpha=1e-3,
        max_iter=200,
        random_state=42,
        early_stopping=True,
        n_iter_no_change=10,
    )


# ──────────────────────────────────────────
# 2. RAGE-QUIT MLP  [9 -> 128 -> 64 -> 1]
# ──────────────────────────────────────────
print("\n[2/4] Training MLP for Rage-Quit Risk...")
mlp_rage = make_mlp().fit(scaler_rage.transform(Xr_train), yr_train)
rage_prob_test = mlp_rage.predict_proba(scaler_rage.transform(Xr_test))[:, 1]
# Labels are sampled from a probability, so even a perfect model can't reach 100% accuracy.
rage_bayes_acc = np.maximum(sigmoid(rage_logit), 1 - sigmoid(rage_logit)).mean()
rage_metrics = {
    "accuracy": round(accuracy_score(yr_test, rage_prob_test >= 0.5), 4),
    "roc_auc": round(roc_auc_score(yr_test, rage_prob_test), 4),
    "best_possible_accuracy": round(float(rage_bayes_acc), 4),
}
print(f"  Converged in {mlp_rage.n_iter_} epochs | {rage_metrics}")

# ──────────────────────────────────────────
# 3. ADDICTION MLP  [10 -> 128 -> 64 -> 3]
# ──────────────────────────────────────────
print("\n[3/4] Training MLP for Addiction Category...")
mlp_addiction = make_mlp().fit(scaler_addiction.transform(Xa_train), ya_train)
add_pred_test = mlp_addiction.predict(scaler_addiction.transform(Xa_test))
addiction_metrics = {
    "accuracy": round(accuracy_score(ya_test, add_pred_test), 4),
    "macro_f1": round(f1_score(ya_test, add_pred_test, average='macro'), 4),
}
print(f"  Converged in {mlp_addiction.n_iter_} epochs | {addiction_metrics}")

# Save models & metadata
joblib.dump(mlp_rage, 'rage_model.pkl')
joblib.dump(mlp_addiction, 'addiction_model.pkl')
joblib.dump(le, 'addiction_label_encoder.pkl')
joblib.dump(scaler_rage, 'scaler_rage.pkl')
joblib.dump(scaler_addiction, 'scaler_addiction.pkl')

dl_metadata = {
    "architecture": "Multi-Layer Perceptron (Dense-128 ReLU -> Dense-64 ReLU -> Softmax), L2 alpha=1e-3",
    "optimizer": "Adam (lr=0.001), early stopping",
    "activation": "ReLU",
    "training_samples": int(len(Xr_train)),
    "test_samples": int(len(Xr_test)),
    "data_source": "Synthetic, population-shaped behavioural data with documented risk-function labels",
    "rage_metrics": rage_metrics,
    "addiction_metrics": addiction_metrics,
    "rage_loss_history": [round(x, 4) for x in mlp_rage.loss_curve_],
    "addiction_loss_history": [round(x, 4) for x in mlp_addiction.loss_curve_],
    "rage_epochs": mlp_rage.n_iter_,
    "addiction_epochs": mlp_addiction.n_iter_,
}

with open('deep_learning_meta.json', 'w') as f:
    json.dump(dl_metadata, f, indent=2)
with open('rage_features.json', 'w') as f:
    json.dump(RAGE_FEATURES, f)
with open('addiction_features.json', 'w') as f:
    json.dump(ADDICTION_FEATURES, f)

print("\n[4/4] Saved: rage_model.pkl, addiction_model.pkl, scalers, label encoder, deep_learning_meta.json")
print("=" * 70)
```

main.py (FastAPI back end)

```
"""
Adaptive Gamer Coaching System — FastAPI Backend
Run: python backend/main.py (or uvicorn main:app --reload --port 8000)
Loads Deep Neural Network (MLP) models & Scalers for Behavioral Analysis
"""

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, Field
import joblib
import json
import numpy as np
import os
from pathlib import Path
from typing import Any, Dict, List, Optional

try:
    from backend.coaching import build_report  # imported as a package (Vercel api/index.py)
    from backend import players
except ImportError:
    from coaching import build_report          # run directly: python backend/main.py
    import players

# ─────────────────────────────────────────────
# PATH SETUP
# ─────────────────────────────────────────────
BACKEND_DIR = Path(__file__).parent
ML_DIR = BACKEND_DIR.parent / "ml"

# ─────────────────────────────────────────────
# LOAD DEEP LEARNING MODELS & SCALERS ON STARTUP
# ─────────────────────────────────────────────
print("Loading Deep Neural Network Models (MLP Architecture)...")

try:
    rage_model = joblib.load(ML_DIR / "rage_model.pkl")
    addiction_model = joblib.load(ML_DIR / "addiction_model.pkl")
    scaler_rage = joblib.load(ML_DIR / "scaler_rage.pkl")
    scaler_addiction = joblib.load(ML_DIR / "scaler_addiction.pkl")
    label_encoder = joblib.load(ML_DIR / "addiction_label_encoder.pkl")

    with open(ML_DIR / "rage_features.json") as f:
        RAGE_FEATURES = json.load(f)
    with open(ML_DIR / "addiction_features.json") as f:
        ADDICTION_FEATURES = json.load(f)
    with open(ML_DIR / "deep_learning_meta.json") as f:
        DL_META = json.load(f)

    print("SUCCESS: All Deep Learning MLP models, scalers, and loss histories loaded")
    MODELS_LOADED = True
except Exception as e:
    print(f"ERROR: Deep Learning model loading failed: {e}")
    MODELS_LOADED = False
    rage_model = addiction_model = scaler_rage = scaler_addiction = label_encoder = None
    RAGE_FEATURES = ADDICTION_FEATURES = []
    DL_META = {}

# ─────────────────────────────────────────────
# COACHING LOGIC
# ─────────────────────────────────────────────
COACHING_TIPS = {
    (True, 'High'): [
        {"text": "Take a 30-min break right now", "category": "break", "icon": "🛑"},
        {"text": "Limit sessions to 2 hours maximum", "category": "gameplay", "icon": "🎮"},
        {"text": "Try box breathing: 4 counts in, hold, 4 out", "category": "mental", "icon": "🧠"},
        {"text": "Talk to someone you trust about how you're feeling", "category": "mental", "icon": "🧠"},
    ],
    (True, 'Medium'): [
```

```
        {"text": "Take a 10-min break after every loss streak", "category": "break", "icon": "🛑"},
        {"text": "Drink water and stretch your hands/neck", "category": "health", "icon": "💧"},
        {"text": "Mute or block toxic players immediately", "category": "gameplay", "icon": "🎮"},
    ],
    (False, 'High'): [
        {"text": "Set a daily gaming time limit and stick to it", "category": "gameplay", "icon": "🎮"},
        {"text": "Replace one gaming session with outdoor activity", "category": "health", "icon": "💧"},
        {"text": "Start tracking your mood before and after sessions", "category": "mental", "icon": "🧠"},
    ],
    (False, 'Medium'): [
        {"text": "Maintain a consistent sleep schedule around gaming", "category": "health", "icon": "💧"},
        {"text": "Mix in casual/co-op games to reduce pressure", "category": "gameplay", "icon": "🎮"},
    ],
    (False, 'Low'): [
        {"text": "Great balance! Keep your sleep schedule consistent", "category": "health", "icon": "💧"},
        {"text": "Stay hydrated during long sessions — water, not energy drinks", "category": "health",
"icon": "💧"},
        {"text": "You're a model gamer. Share your habits with your friends!", "category": "mental", "icon":
"🧠"},
    ],
}

def get_coaching_tips(rage_pred: bool, addiction_category: str) -> List[Dict]:
    key = (rage_pred, addiction_category)
    return COACHING_TIPS.get(key, COACHING_TIPS[(False, 'Low')])

# ──────────────────────────────────────────────
# FASTAPI APP
# ──────────────────────────────────────────────
app = FastAPI(
    title="Adaptive Gamer Coaching System API (Deep Learning Enabled)",
    description="Behavioral Deep Neural Network (MLP) predictions for rage-quit risk and addiction level",
    version="2.0.0"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# ──────────────────────────────────────────────
# REQUEST / RESPONSE MODELS
# ──────────────────────────────────────────────
class PlayerInput(BaseModel):
    stress_level: float = Field(..., ge=1, le=10)
    anxiety_score: float = Field(..., ge=0, le=10)
    daily_gaming_hours: float = Field(..., ge=0, le=24)
    toxic_exposure: float = Field(..., ge=0, le=1)
    night_gaming_ratio: float = Field(..., ge=0, le=1)
    weekly_sessions: int = Field(..., ge=1, le=50)
    sleep_hours: float = Field(..., ge=0, le=16)
    loneliness_score: float = Field(..., ge=0, le=10)
    social_interaction_score: float = Field(..., ge=0, le=10)
    microtransactions_spending: float = Field(..., ge=0)
    years_gaming: int = Field(..., ge=0, le=40)
    happiness_score: float = Field(..., ge=0, le=10)
    depression_score: float = Field(..., ge=0, le=10)
    aggression_score: float = Field(default=5.0, ge=0, le=10)

class CoachingTip(BaseModel):
    text: str
    category: str
    icon: str

class PredictionResponse(BaseModel):
    rage_probability: float
    rage_prediction: bool
```

```
    rage_risk_level: str
    addiction_category: str
    addiction_probabilities: Dict[str, float]
    coaching_tips: List[CoachingTip]
    input_summary: Dict[str, float]
    deep_learning_meta: Dict
    addiction_index: float
    report: Dict[str, Any]

# ──────────────────────────────────────────────
# ENDPOINTS
# ──────────────────────────────────────────────
@app.get("/health")
def health_check():
    return {"status": "ok", "models_loaded": MODELS_LOADED, "architecture": "Deep Multi-Layer Perceptron"}

@app.get("/players/search")
def search_players(q: str = "", limit: int = 8):
    """Search the Valorant player database by Riot ID (name or name#tag)."""
    return {"results": players.search(q, min(limit, 20))}


@app.get("/players/lookup")
def lookup_player(riot_id: str):
    """Full profile for a Riot ID: rank, stats, agents, recent matches and play-time pattern."""
    if not riot_id.strip():
        raise HTTPException(status_code=400, detail="Enter a Riot ID like name#tag")
    return players.lookup(riot_id)


@app.post("/predict", response_model=PredictionResponse)
def predict(player: PlayerInput):
    if not MODELS_LOADED:
        raise HTTPException(status_code=503, detail="Deep Learning models not loaded")

    try:
        x = player.model_dump()

        def rage_batch(rows: List[Dict[str, float]]) -> np.ndarray:
            raw = np.array([[r[f] for f in RAGE_FEATURES] for r in rows])
            return rage_model.predict_proba(scaler_rage.transform(raw))[:, 1]

        def addiction_batch(rows: List[Dict[str, float]]) -> np.ndarray:
            # Expected severity on a 0-1 scale: Low=0, Medium=0.5, High=1
            raw = np.array([[r[f] for f in ADDICTION_FEATURES] for r in rows])
            probs = addiction_model.predict_proba(scaler_addiction.transform(raw))
            weights = np.array([{"Low": 0.0, "Medium": 0.5, "High": 1.0}[c] for c in
label_encoder.classes_])
            return probs @ weights

        def rage_fn(values: Dict[str, float]) -> float:
            return float(rage_batch([values])[0])

        def addiction_probs_fn(values: Dict[str, float]) -> Dict[str, float]:
            raw = np.array([[values[f] for f in ADDICTION_FEATURES]])
            probs = addiction_model.predict_proba(scaler_addiction.transform(raw))[0]
            return {cls: float(p) for cls, p in zip(label_encoder.classes_, probs)}

        def addiction_index_fn(values: Dict[str, float]) -> float:
            return float(addiction_batch([values])[0])

        def addiction_class_fn(values: Dict[str, float]) -> str:
            p = addiction_probs_fn(values)
            return max(p, key=p.get)

        rage_prob = rage_fn(x)
        rage_pred = rage_prob >= 0.5

        if rage_prob < 0.40:
            rage_risk_level = "LOW"
```

```
        elif rage_prob < 0.70:
            rage_risk_level = "MEDIUM"
        else:
            rage_risk_level = "HIGH"

        add_probs = addiction_probs_fn(x)
        add_category = max(add_probs, key=add_probs.get)
        add_index = addiction_index_fn(x) * 100

        report = build_report(
            x, rage_fn, addiction_index_fn, addiction_class_fn, rage_batch, addiction_batch,
            rage_prob, rage_risk_level, add_category, add_index,
        )

        tips = get_coaching_tips(rage_pred, add_category)

        input_summary = {
            "stress_level": player.stress_level,
            "anxiety_score": player.anxiety_score,
            "loneliness_score": player.loneliness_score,
            "gaming_intensity": min(10, player.daily_gaming_hours / 1.2),
            "sleep_deprivation": max(0, 10 - player.sleep_hours),
            "social_score": player.social_interaction_score,
        }

        return PredictionResponse(
            rage_probability=round(rage_prob, 4),
            rage_prediction=rage_pred,
            rage_risk_level=rage_risk_level,
            addiction_category=add_category,
            addiction_probabilities=add_probs,
            coaching_tips=[CoachingTip(**tip) for tip in tips],
            input_summary={k: round(v, 2) for k, v in input_summary.items()},
            deep_learning_meta=DL_META,
            addiction_index=round(add_index, 1),
            report=report,
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Prediction error: {str(e)}")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

coaching.py (Shapley attribution, plan targets, taper schedule, warning signs)

```
def attribute(x: Dict[str, float], rage_batch: Callable, addiction_batch: Callable, n_perm: int = 160) ->
List[Dict]:
    """Sampled Shapley values against a healthy baseline profile.

    For random orderings of the factors we move the profile from the healthy baseline to the
    player's values one factor at a time and record how much each step changes each model's
    output. Averaged over orderings, the contributions sum exactly to
    score(player) - score(healthy baseline), so they stay meaningful even when a model is
    saturated near 0% or 100%. Values are in percentage points."""
    feats = list(FEATURES)
    baseline = dict(x)
    for f in feats:
        baseline[f] = FEATURES[f]["baseline"]

    rng = np.random.default_rng(7)
    points, orders = [], []
    for _ in range(n_perm):
        order = rng.permutation(len(feats))
        orders.append(order)
        cur = dict(baseline)
        points.append(dict(cur))
        for i in order:
            cur[feats[i]] = x[feats[i]]
```

```
            points.append(dict(cur))

    rage_vals = np.asarray(rage_batch(points))
    add_vals = np.asarray(addiction_batch(points))
    rage_contrib = np.zeros(len(feats))
    add_contrib = np.zeros(len(feats))
    step = len(feats) + 1
    for k, order in enumerate(orders):
        r = rage_vals[k * step:(k + 1) * step]
        a = add_vals[k * step:(k + 1) * step]
        for j, i in enumerate(order):
            rage_contrib[i] += r[j + 1] - r[j]
            add_contrib[i] += a[j + 1] - a[j]
    rage_contrib = rage_contrib / n_perm * 100
    add_contrib = add_contrib / n_perm * 100

    rows = []
    for i, feat in enumerate(feats):
        meta = FEATURES[feat]
        value = x[feat]
        why, steps = COPY[feat](value, x)
        rows.append({
            "feature": feat,
            "label": meta["label"],
            "group": meta["group"],
            "value": value,
            "display_value": fmt(feat, value),
            "healthy_range": list(meta["healthy"]),
            "healthy_label": fmt_range(feat),
            "scale": [meta["min"], meta["max"]],
            "worse": meta["worse"],
            "status": status_of(feat, value),
            "tilt_impact": round(float(rage_contrib[i]), 1),
            "addiction_impact": round(float(add_contrib[i]), 1),
            "why": why,
            "steps": steps,
        })
    rows.sort(key=lambda r: max(r["tilt_impact"], r["addiction_impact"]), reverse=True)
    return rows


# ──────────────────────────────────────────────
# PLAN TARGETS & PROJECTION
# Only the habits a player directly controls are changed for the projection.
# ──────────────────────────────────────────────
def plan_targets(x: Dict[str, float]) -> Dict[str, float]:
    t = dict(x)
    t["daily_gaming_hours"] = min(x["daily_gaming_hours"], 3.0)
    t["weekly_sessions"] = min(x["weekly_sessions"], 14)
    t["night_gaming_ratio"] = min(x["night_gaming_ratio"], 0.15)
    t["sleep_hours"] = max(x["sleep_hours"], 7.5)
    t["toxic_exposure"] = min(x["toxic_exposure"], 0.2)
    t["microtransactions_spending"] = min(x["microtransactions_spending"], 30)
    return t


def taper_schedule(hours: float, target: float = 3.0) -> List[Dict]:
    """Week-by-week daily-hours targets, cutting about 20% a week (at least 30 min) until target."""
    weeks = [{"label": "Now", "hours": round(hours, 1)}]
    h = hours
    w = 1
    while h > target and w <= 8:
        h = max(target, round(min(h * 0.8, h - 0.5) * 2) / 2)
        weeks.append({"label": f"Wk {w}", "hours": h})
        w += 1
    return weeks


def warning_signs(x: Dict[str, float]) -> List[Dict]:
    """Behavioural patterns that overlap with WHO gaming-disorder features. Not a diagnosis."""
```

```
    signs = [
        ("Hard to control how long you play", x["daily_gaming_hours"] >= 6 or x["weekly_sessions"] >= 30),
        ("Gaming pushing out sleep", x["sleep_hours"] < 6.5 and x["night_gaming_ratio"] >= 0.4),
        ("Gaming taking priority over people", x["social_interaction_score"] <= 3 and
x["daily_gaming_hours"] >= 4),
        ("Playing on despite feeling worse", x["depression_score"] >= 6 and x["daily_gaming_hours"] >= 4),
        ("Spending beyond a hobby budget", x["microtransactions_spending"] >= 150),
    ]
    return [{"label": label, "present": bool(present)} for label, present in signs]


SUPPORT_RESOURCES = [
    {"name": "Tele-MANAS (India)", "detail": "Free 24/7 mental-health helpline: call 14416 or 1-800-891-
4416"},
    {"name": "NIMHANS SHUT Clinic (Bengaluru)", "detail": "Service for Healthy Use of Technology: specialist
help for gaming and internet overuse"},
    {"name": "988 Lifeline (US)", "detail": "Call or text 988, available 24/7"},
    {"name": "Samaritans (UK & Ireland)", "detail": "Call 116 123, free and available 24/7"},
    {"name": "Find a Helpline", "detail": "findahelpline.com lists free, confidential lines in your
country"},
]
```

Verdict.jsx (verdict lower-third)

```
import { ArrowRight, MoveRight } from 'lucide-react';

const tiltTone = (p) => (p < 0.4 ? 'ok' : p < 0.7 ? 'watch' : 'risk');
const addTone = (c) => ({ Low: 'ok', Medium: 'watch', High: 'risk' }[c]);
const pct = (p) => Math.round(p * 100);
const cap = (s) => s.charAt(0).toUpperCase() + s.slice(1).toLowerCase();

function Line({ lane, children, note }) {
  return (
    <div className="vd-line">
      <dt>{lane}</dt>
      <dd className="vd-figure">{children}</dd>
      <dd className="vd-note">{note}</dd>
    </div>
  );
}

// The lower-third: a headline, one deck line, then three ruled data lines that carry
// the numbers (where you stand, what drives it, what the plan does) instead of a paragraph.
export default function Verdict({ data }) {
  const { summary, projected } = data.report;
  const drivers = summary.drivers || [];
  const tilt = data.rage_probability;
  const idx = Math.round(data.addiction_index);
  const cat = data.addiction_category;
  // Only show the before/after when the plan moves a score by a few points; otherwise it's noise.
  const changes = projected.changes.length > 0
    && (pct(tilt) - pct(projected.tilt_probability) >= 3 || idx - Math.round(projected.addiction_index) >=
3);

  return (
    <section className="verdict" aria-labelledby="verdict-title">
      <div className="verdict-plate">
        <h2 id="verdict-title" className="verdict-headline">{summary.headline}</h2>
        <p className="verdict-deck">{summary.deck}</p>

        <dl className="vd-lines">
          <Line lane="Now" note={`${cap(data.rage_risk_level)} tilt risk, ${cat.toLowerCase()} addiction
level.`}>
            <span className={`vd-stat ink-${tiltTone(tilt)}`}>{pct(tilt)}<small>% tilt</small></span>
            <span className={`vd-stat ink-${addTone(cat)}`}>{idx}<small>/100 addiction</small></span>
          </Line>

          <Line
            lane="Driving it"
```

```
            note={drivers.length ? (drivers.length > 1 ? 'Fix these two first. They carry most of your
risk.' : 'Fix this first. It carries most of your risk.') : 'No habit is pushing your risk up
meaningfully.'}
          >
            {drivers.length ? drivers.map((d) => (
              <span key={d.feature} className="vd-driver">
                {d.label}<b className="num">+{d.points}</b>
              </span>
            )) : <span className="vd-word ink-ok">Nothing major</span>}
          </Line>

          <Line
            lane="With the plan"
            note={changes ? 'Same models, re-run on the plan’s targets.' : 'You’re already at or near every
healthy target.'}
          >
            {changes ? (
              <>
                <span className="vd-shift">
                  <s className="num">{pct(tilt)}</s><MoveRight size={16} aria-label="to" />
                  <span className={`vd-stat ink-${tiltTone(projected.tilt_probability)}`}>
{pct(projected.tilt_probability)}<small>% tilt</small></span>
                </span>
                <span className="vd-shift">
                  <s className="num">{idx}</s><MoveRight size={16} aria-label="to" />
                  <span className={`vd-stat ink-${addTone(projected.addiction_category)}`}>
{Math.round(projected.addiction_index)}<small>/100</small></span>
                </span>
              </>
            ) : <span className="vd-word ink-ok">Hold steady</span>}
          </Line>
        </dl>
      </div>
      <div className="verdict-next">
        <ArrowRight size={22} aria-hidden className="verdict-next-icon" />
        <p>
          <strong>Do this first.</strong> {summary.next_action}
        </p>
      </div>
    </section>
  );
}
```

5.2 Screenshots

The figures below show the running application for the player *kyuu#0711*.

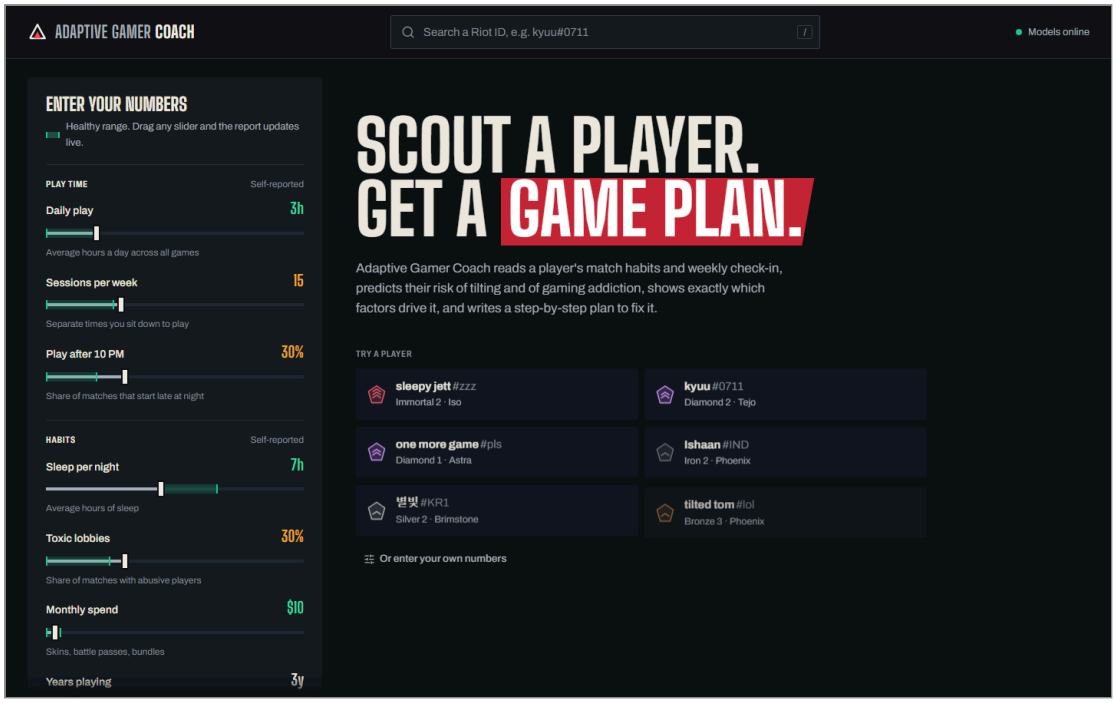


Fig 4: Home page with player search



Fig 5: Player profile and report overview

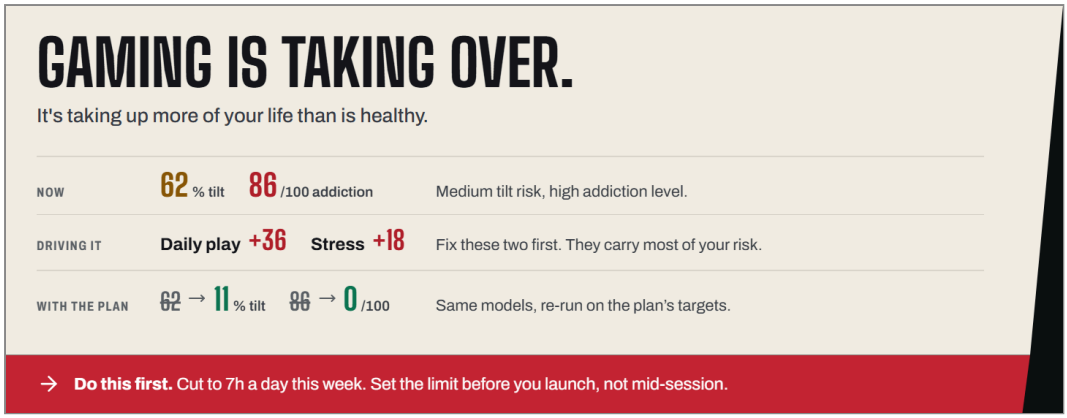


Fig 6: Verdict lower-third

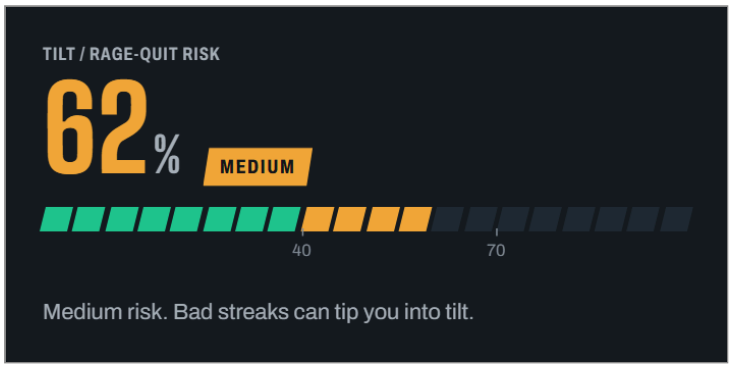


Fig 7: Tilt / rage-quit score

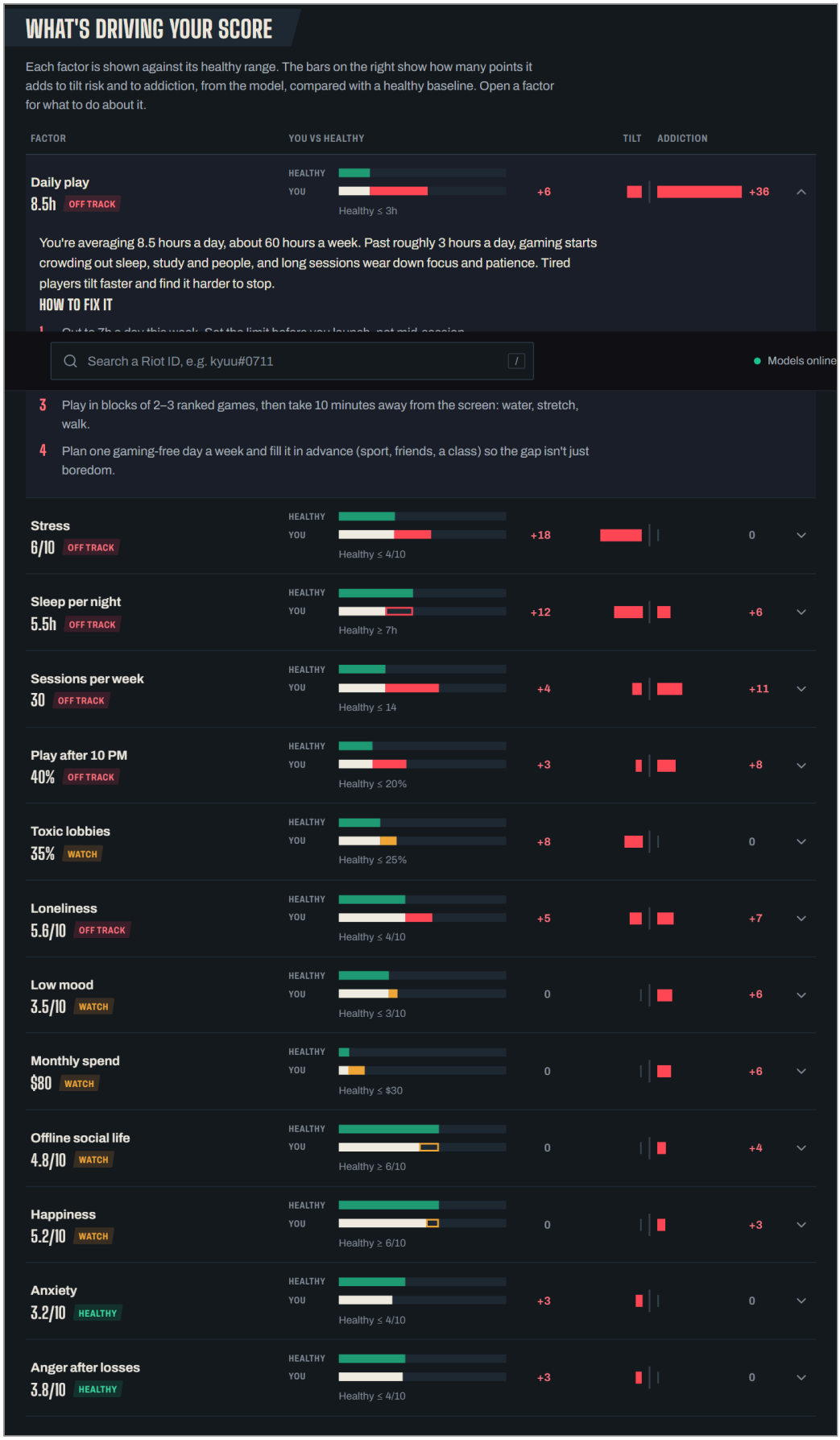

WHAT'S DRIVING YOUR SCORE
Each factor is shown against its healthy range. The bars on the right show how many points it adds to tilt risk and to addiction, from the model, compared with a healthy baseline. Open a factor for what to do about it.
FACTOR
YOU VS HEALTHY
TILT
ADDICTION
Daily play
8.5h
OFF TRACK
HEALTHY
YOU
+6
+36
Healthy ≤ 3h
You're averaging 8.5 hours a day, about 60 hours a week. Past roughly 3 hours a day, gaming starts crowding out sleep, study and people, and long sessions wear down focus and patience. Tired players tilt faster and find it harder to stop.
HOW TO FIX IT
Search a Riot ID, e.g. kyuu#0711
/
Models online
3 Play in blocks of 2–3 ranked games, then take 10 minutes away from the screen: water, stretch, walk.
4 Plan one gaming-free day a week and fill it in advance (sport, friends, a class) so the gap isn't just boredom.
Stress
6/10
OFF TRACK
HEALTHY
YOU
+18
0
Healthy ≤ 4/10
Sleep per night
5.5h
OFF TRACK
HEALTHY
YOU
+12
+6
Healthy ≥ 7h
Sessions per week
30
OFF TRACK
HEALTHY
YOU
+4
+11
Healthy ≤ 14
Play after 10 PM
40%
OFF TRACK
HEALTHY
YOU
+3
+8
Healthy ≤ 20%
Toxic lobbies
35%
WATCH
HEALTHY
YOU
+8
0
Healthy ≤ 25%
Loneliness
5.6/10
OFF TRACK
HEALTHY
YOU
+5
+7
Healthy ≤ 4/10
Low mood
3.5/10
WATCH
HEALTHY
YOU
0
+6
Healthy ≤ 3/10
Monthly spend
$80
WATCH
HEALTHY
YOU
0
+6
Healthy ≤ $30
Offline social life
4.8/10
WATCH
HEALTHY
YOU
0
+4
Healthy ≥ 6/10
Happiness
5.2/10
WATCH
HEALTHY
YOU
0
+3
Healthy ≥ 6/10
Anxiety
3.2/10
HEALTHY
HEALTHY
YOU
+3
0
Healthy ≤ 4/10
Anger after losses
3.8/10
HEALTHY
HEALTHY
YOU
+3
0
Healthy ≤ 4/10

Fig 8: What is driving the score (factor board)



Fig 9: Now versus after the plan



Fig 10: Recovery plan and step-down schedule

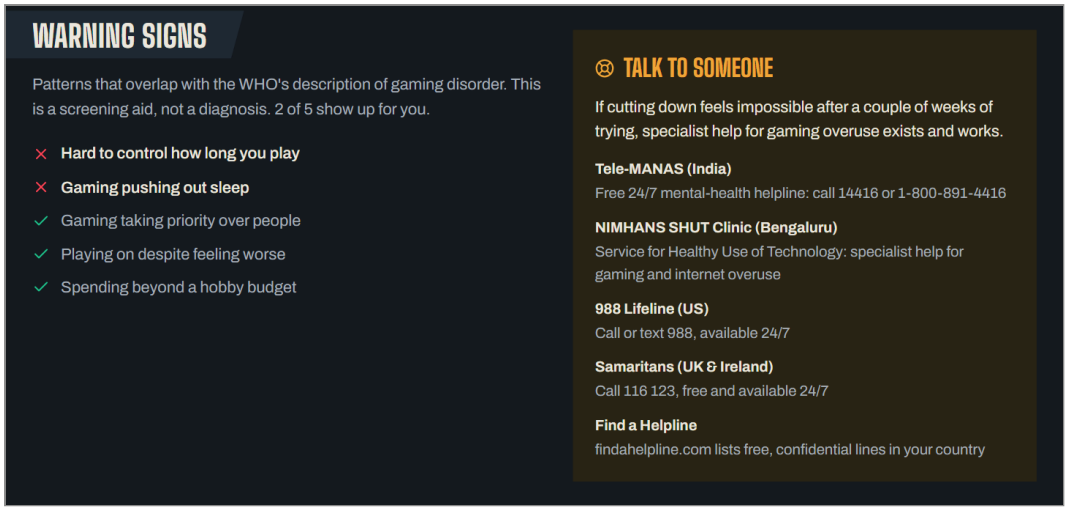


Fig 11: Warning signs and support



Fig 12: Behaviour and check-in sliders

CHAPTER-6
RESULTS & CONCLUSION

6.1 Results

The system was built, trained and run end to end. The two neural networks were evaluated on a held-out test set of 4,400 profiles that were never used for training. Table 7 lists the results.

Table 7: Model Performance Evaluation

Model	Metric	Performance
Tilt (rage-quit) MLP	Accuracy	79.7%
Tilt (rage-quit) MLP	ROC-AUC	0.86
Tilt (rage-quit) MLP	Best achievable accuracy (noise ceiling)	79.3%
Addiction MLP	Accuracy	92.6%
Addiction MLP	Macro F1-score	0.91
Prediction + explanation (API)	Response time per request	about 0.1–0.3 s (includes 160 Shapley orderings)

The tilt labels are sampled from a probability, so even a perfect model cannot be right every time; the noise ceiling for this data is 79.3% and the network reaches 79.7%, within about one point of it, with an ROC-AUC of 0.86. The addiction network separates the Low, Medium and High classes with 92.6% accuracy. Figure 13 shows the training loss falling smoothly for both networks before early stopping ended training.

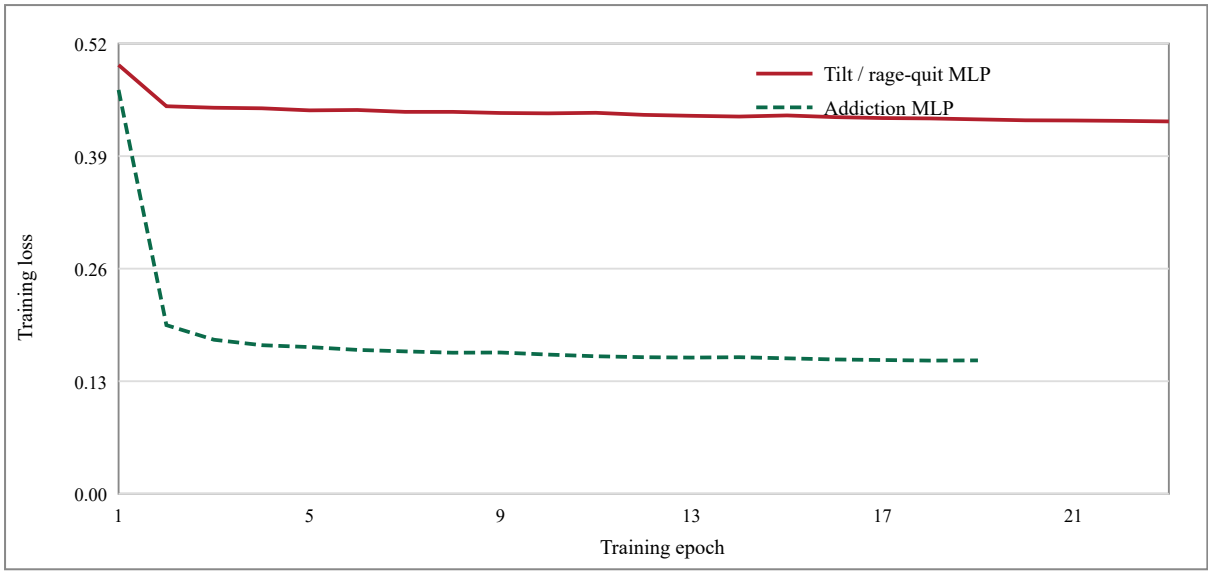


Fig 13: Training loss curves for the two networks

Beyond the metrics, the checks that matter for a coaching tool were verified directly. The scores shown in the verdict match the scores shown in the score cards for all 144 players; every slider the user can move changes the prediction in the expected direction; healthy players receive a “Healthy balance” verdict with no alarming advice, while a player such as kyuu#0711 (8.5 hours a day, 5.5 hours of sleep) sees a tilt probability of 62%, an addiction index of 86 out of 100 and daily play and stress named as the two main drivers. For that player, re-running the models on the plan's targets (3 hours a day, 14 sessions a week, 7.5 hours of sleep) projects the tilt probability falling from 62% to 11% and the addiction index from 86 to 0, which the interface shows next to the current values.

The implemented system delivers: player search over 144 Valorant profiles; a complete player card; a verdict that can be read in a few seconds; two risk scores with confidence; a factor board with per-factor points for tilt and addiction; a phased recovery plan with a taper schedule; a projected outcome; WHO-based warning signs and helplines; a responsive layout that works on phones; and a polished load-in animation.

6.2 Conclusion

The Adaptive Gamer Coaching System shows that machine learning can be applied to the gap between high-stress competitive gaming and healthy habits in a way that a player can understand and act on. Two Multi-Layer Perceptrons predict tilt and addiction risk from thirteen everyday factors, a Shapley-value layer explains each prediction in points, and a coaching engine turns the explanation into a verdict, a phased plan and a projected improvement. The result is a tool that tells a player not only how much risk they carry but why, and what to change first.

The project also demonstrates good engineering practice: a clean split between a React front end and a FastAPI back end, validated inputs, pinned model dependencies, reproducible training and a design system that keeps the interface calm and readable. Because the training data is synthetic, the system is a screening and coaching aid, not a diagnostic tool, and the interface says so wherever it matters.

Future enhancements

- **Live game data.** Connect the Riot Games API so that real match history and play-time patterns replace the synthetic player database.
- **Real survey data.** Validate and retrain the models on real, consented survey datasets such as the Global Gaming Mental Health Survey, and measure calibration against clinical scales.
- **Biometrics.** Add heart rate and galvanic skin response from wearables for a better read on in-match emotion.
- **Mobile app.** Provide a mobile client that continuously captures wellbeing and sends break reminders.
- **Conversational coaching.** Add an LLM layer for richer, adaptive coaching that follows the player's stated preferences and tone.
- **More games.** Extend the API to other game genres so that risk can be tracked across all of a player's games.

CHAPTER-7
REFERENCES

[1] S. C. Cregan, A. J. Toth, and M. J. Campbell, "Playing for keeps or just playing with emotion? Studying tilt and emotion regulation in video games", Front. Psychol., vol. 15, Art. no. 1385242, Apr. 2024.

[2] M. N. Alan et al., "#GAMEADDICTED: A Machine Learning Framework for DGA Detection", IEEE Access, vol. 13, pp. 136-150, 2025.

[3] A. Alrobai et al., "Online Peer Support Groups for Countering Digital Game Addiction", in Proc. Int. Conf. on Digital Health, 2025, pp. 150-155.

[4] S. Ali et al., "Software Industry Awareness of Digital Addiction Consequences", J. Softw. Eng. Res. Dev., vol. 12, no. 3, pp. 149-158, 2025.

[5] R. Ahmed et al., "Forecasting Gaming Addiction Likelihood Among Youth in Bangladesh using Machine Learning", in Proc. 6th Int. Conf. on I-SMAC, 2024, pp. 152-160.

[6] A. Weinstein, "Computer and Video Game Addiction—A Comparison between Users and Non-Users", Am. J. Drug Alcohol Abuse, vol. 36, no. 5, pp. 154-159, 2010.

[7] J. Knol et al., "Towards Explainable Prediction of Player Frustration in Video Games", in Proc. Found. Digit. Games (FDG), 2021, pp. 164-172.

[8] K. Karpouzis and G. N. Yannakakis, "Affective Game Computing: A Survey", IEEE Trans. Games, vol. 10, no. 1, pp. 180-195, 2018.

[9] M. J. Canossa et al., "Detecting Social Interaction Quality in Games through Behavioral Traces", in Proc. ACM/IEEE Frontiers in Educ., 2019, pp. 196-204.

[10] L. E. Nacke et al., "BrainHex: A Neurobiological Gamer Typology Survey", Entertain. Comput., vol. 5, no. 1, pp. 200-210, 2014.

[11] J. E. Beck and Y. Gong, "Wheel-spinning: Students who fail to master a skill despite large amounts of practice", in Proc. Artif. Intell. Educ., 2013, pp. 217-226.

[12] N. S. Williams et al., "On Quitting: Performance and Practice in Online Game Play", Psychol. Res., vol. 85, pp. 112-124, 2021.

[13] J. Lee et al., "Digital Empathy and Nursing Students: The Impact of Internet Game Addiction", J. Nurs. Educ. Pract., vol. 11, no. 2, pp. 148-155, 2021.

[14] R. Kharisma et al., "Online Game Addiction and Sleep Quality among College Students", Int. J. Psychosoc. Rehabil., vol. 24, no. 8, pp. 147-152, 2020.

[15] L. Bezençon et al., "Escape Addict: A Digital Escape Room for Addiction Prevention", in Proc. CHI Play, 2022, pp. 149-153.

[16] Y. Li et al., "Mendelian Randomization of Computer Games and Mental Health: A Genetic Perspective", Nature Mental Health, vol. 1, pp. 148-154, 2022.

[17] A. Kornev et al., "Gaming Behavior and Brain Activation: An fNIRS and Machine Learning Study", Brain Behav., vol. 12, no. 4, pp. 145-153, 2022.

[18] M. Rahman et al., "User Perspective Bangla Sentiment Analysis for Online Gaming Addiction", in Proc. 4th Int. Conf. on Comput. Intell., 2023, pp. 156-162.

[19] Z. Xu et al., "Research on Lexical Data Set Analysis for Addiction based on Decision Tree Models", Highlights Sci. Eng. Technol., vol. 28, pp. 156-160, 2024.

[20] H. Wang et al., "Using Bayesian Networks to Predict At-Risk Individuals for Addiction", in Proc. Int. Conf. on Data Sci., 2023, pp. 156-161.

[21] J. S. Cheah et al., "Motives to Play Digital Games: A Systematic Review and Meta-Analysis", Comput. Hum. Behav., vol. 120, Art. no. 106756, 2021.

[22] A. J. van Rooij, D. J. Kuss, M. D. Griffiths, G. W. Shorter, T. M. Schoenmakers, and D. van de Mheen, "The (co-)occurrence of problematic video gaming, substance use, and psychosocial problems in adolescents", J. Behav. Addict., vol. 3, no. 3, pp. 157–165, Sep. 2014.

[23] A. Esteve et al., "Dual Diagnosis in Adolescents with Problematic Use of Video Games: Beyond Substances", Brain Sci., vol. 12, no. 8, Art. no. 1110, Aug. 2022.

[24] T. L. Burleigh, M. D. Griffiths, A. Sumich, V. Stavropoulos, and D. J. Kuss, "A Systematic Review of the Co-occurrence of Gaming Disorder and Other Potentially Addictive Behaviors", Curr. Addict. Rep., vol. 6, no. 4, pp. 383–401, Dec. 2019.

[25] N. Rismayanti, "Predicting Online Gaming Behaviour Using Machine Learning Techniques", Indones. J. Data Sci., vol. 5, no. 2, pp. 166-173, July 2024.

[26] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions", in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[27] L. S. Shapley, "A Value for n-Person Games", in Contributions to the Theory of Games, vol. II, Princeton University Press, 1953, pp. 307-317.

[28] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization", in Proc. Int. Conf. on Learning Representations (ICLR), 2015.

[29] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[30] World Health Organization, "ICD-11 for Mortality and Morbidity Statistics: 6C51 Gaming disorder", 2019.